

ספר פרויקט - מערכת ניהול חנות

תוכנה לניהול מלאי, מכירות והזמנות לקוחות

StorePilot

Store Management & E-Commerce Android Application

Project Book

Final Year Computer Science Project

Student:	Ahmad Hijazy
Email:	ahmadhijazy480@gmail.com
School:	High School - Computer Science Track
Year:	2025 / 2026
Language:	Java (Android)
Backend:	Firebase Firestore + Firebase Auth

Submitted in partial fulfillment of final project requirements

פרויקט גמר - תכנות Android

מוגש לצורך סיום תכנית הלימודים במדעי המחשב

Table of Contents

1 Introduction

- 1.1 Background and Motivation
- 1.2 Project Objectives
- 1.3 Target Users
- 1.4 Technology Stack

2 System Requirements

- 2.1 Functional Requirements
- 2.2 Non-Functional Requirements

3 System Architecture

- 3.1 MVVM Design Pattern
- 3.2 Firebase Integration
- 3.3 Firestore Data Collections
- 3.4 Role-Based Access Control

4 Data Model

- 4.1 - 4.8 Entity Classes (User, Product, Order, Sale, Task, Season, CartItem, SupportMessage)

5 Implementation - Core Modules

- 5.1 StorePilotApp | 5.2 SessionManager | 5.3 PermissionManager
- 5.4 FirestoreManager | 5.5 LowStockReceiver | 5.6 NotificationHelper
- 5.7 CryptoUtil | 5.8 BaseActivity & FirebaseAuthHelper

6 Implementation - Repositories

- 6.1 ProductRepository | 6.2 UserRepository | 6.3 OrderRepository
- 6.4 SaleRepository | 6.5 TaskRepository | 6.6 SeasonRepository
- 6.7 SupportRepository

7 Implementation - ViewModels & UI

- 7.1 AuthViewModel & LoginActivity
- 7.2 ProductViewModel & ProductListFragment
- 7.3 OrderViewModel & CheckoutFragment
- 7.4 DashboardFragment

8 Screen Flow & Navigation

9 Testing

A Appendix: Full Source Code (13 classes)

Chapter 1: Introduction

1.1 Background and Motivation

Small and medium-sized retail stores often struggle to manage their operations efficiently. Tasks such as tracking inventory, recording sales, managing customer orders, and scheduling staff are frequently handled through paper records or disconnected spreadsheets. This results in errors, wasted time, and poor visibility into the store's real-time state.

StorePilot was developed to solve this problem by providing a single, unified Android application that digitizes all core store management functions. The application targets clothing and general retail stores and was designed with simplicity in mind, so that staff at all technical levels can adopt it quickly.

1.2 Project Objectives

- Provide a real-time inventory management system with automatic low-stock notifications.
- Enable staff to record walk-in sales and track revenue by day, week, and month.
- Allow customers to browse products and place online orders from their smartphones.
- Implement a task management module for assigning and tracking staff responsibilities.
- Manage seasonal product collections with configurable end-of-season dashboard alerts.
- Provide a live support chat between customers and store managers.
- Enforce role-based access control so each staff role sees only what they need.
- Synchronize all data in real time via Firebase Firestore across multiple devices.

1.3 Target Users

Owner

Full administrative control. Manages users, products, views all data.

Store Manager

Manages inventory, records sales, manages tasks and customer orders.

Shift Manager

Records sales, views products, manages team-level tasks.

Employee

Records sales, views products, creates private personal tasks.

Customer

Browses products, adds to cart, places orders, tracks delivery, chats.

1.4 Technology Stack

Platform:	Android (API 24+), Java
Architecture:	MVVM with LiveData (Google recommended pattern)
UI Toolkit:	Material Design, RecyclerView, CardView, BottomNavigationView
Authentication:	Firebase Authentication (email/password)
Database:	Firebase Cloud Firestore (real-time NoSQL cloud database)
Notifications:	AlarmManager + BroadcastReceiver + NotificationManager
IDE:	Android Studio
Build Tool:	Gradle (Groovy DSL)
Min SDK:	API 24 (Android 7.0 Nougat)
Target SDK:	API 34 (Android 14)

Chapter 2: System Requirements

2.1 Functional Requirements

FR-01 - User Registration & Login

Staff and customers register with email/password and log in via Firebase Auth. The system routes them to different UIs based on their role field.

FR-02 - Role-Based Access

Every screen and action is gated by PermissionManager. Unauthorised users cannot see or trigger features outside their role.

FR-03 - Product Inventory

Authorised staff create, view, edit, and delete products. Each product has name, category, size, colour, quantity, selling price, cost price, and image URL.

FR-04 - Low-Stock Alerts

AlarmManager triggers LowStockReceiver hourly. When quantity ≤ 5 , a system notification is posted automatically.

FR-05 - Sales Recording

Employees record manual walk-in sales (product, quantity, price, date, staff).

FR-06 - Customer E-Commerce

Customers browse products, add to Firestore cart, mark favourites, place orders with address and payment method.

FR-07 - Order Management

Managers view active orders and progress status: PENDING -> PROCESSING -> SHIPPED -> DELIVERED. DELIVERED auto-creates a Sale record.

FR-08 - Task Management

Staff create tasks with title, description, assignee, priority, due date. Tasks can be private or team-wide.

FR-09 - Season Management

Managers define seasonal collections. Dashboard banner appears when a season ends within 30 days.

FR-10 - Support Chat

Customers open a live chat with managers. Messages stored in support/{customerId}/messages Firestore sub-collection.

FR-11 - User Management

Owners view all registered users and their roles via the Admin screen.

2.2 Non-Functional Requirements

Performance:	Firestore queries return results within 3 seconds on 4G.
Usability:	Material Design guidelines. Bottom navigation for staff; separate Activity for customers.
Security:	Firebase Auth manages credentials. No raw passwords stored. CryptoUtil provides PBKDF2WithHmacSHA256 hashing.
Scalability:	Firestore scales automatically. Real-time listeners keep all connected clients in sync within seconds.
Offline Mode:	Firestore SDK caches recent reads. Writes are queued and synced when connectivity resumes.
Compatibility:	Android 7.0+ (API 24+), covering more than 95% of active devices.
Notifications:	NotificationChannel required on API 26+. POST_NOTIFICATIONS permission requested on Android 13+.

Chapter 3: System Architecture

3.1 MVVM Design Pattern

StorePilot follows the Model-View-ViewModel (MVVM) architecture recommended by Google for Android. This pattern separates the application into three layers:

Model

Data entities (User, Product, Order, etc.) and Repositories. Repositories wrap all Firestore calls and expose LiveData so the rest of the app is decoupled from the data source.

ViewModel

AndroidViewModel subclasses that hold and transform data for the UI. They survive configuration changes (e.g. rotation) because they are scoped to the Activity/Fragment lifecycle, not the View.

View

Activities and Fragments observe LiveData from ViewModels. The UI updates automatically when data changes, with no manual refresh.

Typical user-action flow:

User taps "Add Sale" -> ViewModel.insert(sale) -> Repository.insert(sale) -> Firestore.collection("sales").add(...) -> Snapshot listener fires -> LiveData updated -> UI refreshes automatically.

3.2 Firebase Integration

Firebase Authentication

Handles sign-up and sign-in with email/password. After sign-in, the Firebase UID fetches the user profile from Firestore "users". FirebaseAuthHelper wraps createUserWithEmailAndPassword() and signInWithEmailAndPassword() behind a simple AuthCallback interface.

Firebase Cloud Firestore

All application data lives in Firestore. Every repository sets up addSnapshotListener() in its constructor so changes propagate to the UI in real time. Batch writes (WriteBatch) are used during order placement to atomically update 4 collections in a single transaction.

3.3 Firestore Data Collections

users	Keyed by Firebase UID. Profile + role.
products	All store products with quantity, price, cost.

sales	Manual sales recorded by staff.
orders	Customer orders. Sub-collection "items" holds line items.
tasks	Staff tasks with assignee, priority, status.
seasons	Seasonal collections with start/end dates.
carts	carts/{customerId}/items/{productId} - persistent cart.
favorites	favorites/{customerId}/items/{productId} - wish list.
support	support/{customerId}/messages - chat history.

3.4 Role-Based Access Control

PermissionManager holds a static Map<String, List<String>> mapping each permission constant to the roles that hold it. Every UI element that requires elevated access calls PermissionManager.currentUserHasPermission() before rendering. BaseActivity.hideViewIfUnauthorized() wraps this for Activities.

Permission	OWNER	MGR	SHIFT	EMP	CUST
MANAGE_USERS	Y	-	-	-	-
MANAGE_PRODUCTS	Y	-	-	-	-
VIEW_PRODUCTS	Y	Y	Y	Y	-
CREATE_SALE	Y	Y	Y	Y	-
VIEW_SALES_HISTORY	Y	Y	Y	-	-
MANAGE_SEASONS	Y	Y	-	-	-
CREATE_PRIVATE_TASK	Y	Y	Y	Y	-
VIEW_TEAM_TASKS	Y	Y	Y	-	-
MANAGE_TASKS	Y	Y	-	-	-
VIEW_ADMIN	Y	-	-	-	-

Chapter 4: Data Model

Each entity is a plain Java object (POJO) with no ORM annotations - all persistence is handled directly via Firestore. Every entity provides:

- Default no-arg constructor (required by Firestore deserialization).
- toMap() method converting the object to Map<String,Object>.
- Static fromDoc(DocumentSnapshot) factory for reading from Firestore.
- JavaBeans-style getter methods.

4.1 User

Package: com.storepilot.db.entities

Represents both staff and customers. The role field determines permissions.

Field Name	Type	Scope	Purpose
id	int	public	Local numeric ID (derived from createdAt)
uid	String	public	Firebase Auth UID (Firestore document key)
fullName	String	public	Display name
username	String	public	Unique username
email	String	public	Email used for Firebase Auth
phone	String	public	Contact phone
role	String	public	OWNER STORE_MANAGER SHIFT_MANAGER EMPLOYEE CUSTOMER
createdAt	long	public	Unix timestamp (ms) of account creation

4.2 Product

Package: com.storepilot.db.entities

Represents a store product. quantity triggers the low-stock alert when ≤ 5 .

Field Name	Type	Scope	Purpose
id	int	public	Local numeric ID
firestoreId	String	public	Firestore auto-generated document ID
name	String	public	Product name
category	String	public	Category (e.g. Shirts, Pants)
size	String	public	Size code (S, M, L, XL)
color	String	public	Colour description
quantity	int	public	Stock level - alert fires when ≤ 5
price	double	public	Selling price per unit (USD)

costPrice	double	public	Purchase cost per unit
imageUrl	String	public	Optional product image URL
createdAt	long	public	Unix timestamp (ms)

4.3 Order

Package: com.storepilot.db.entities

Customer order. Status: PENDING -> PROCESSING -> SHIPPED -> DELIVERED.

Field Name	Type	Scope	Purpose
id	int	public	Local numeric ID
firestoreId	String	public	Firestore document ID
customerId	int	public	References User.id of the customer
totalPrice	double	public	Total order value
status	String	public	PENDING PROCESSING SHIPPED DELIVERED CANCELLED
createdAt	long	public	Unix timestamp (ms)
paymentMethod	String	public	CASH_ON_DELIVERY PAYPAL CREDIT_CARD
shippingAddress	String	public	Delivery address

4.4 Sale

Package: com.storepilot.db.entities

Records a sale transaction. Created by staff or auto-generated on order delivery.

Field Name	Type	Scope	Purpose
id	int	public	Local numeric ID
firestoreId	String	public	Firestore document ID
productId	Integer	public	References Product.id (nullable)
quantity	int	public	Units sold
totalPrice	double	public	Revenue for this sale
saleDate	long	public	Unix timestamp (ms)
soldBy	Integer	public	References User.id of staff (nullable)
notes	String	public	Optional notes

4.5 Task

Package: com.storepilot.db.entities

Work task with Kanban-style status and privacy flag.

Field Name	Type	Scope	Purpose
------------	------	-------	---------

id	int	public	Local numeric ID
firestoreId	String	public	Firestore document ID
title	String	public	Short task title
description	String	public	Detailed description
assignedTo	Integer	public	References User.id of assignee
createdBy	Integer	public	References User.id of creator
status	String	public	TODO IN_PROGRESS DONE
priority	String	public	LOW MEDIUM HIGH
isPrivate	boolean	public	Visible only to creator when true
dueDate	long	public	Unix timestamp (ms)
createdAt	long	public	Unix timestamp (ms)

4.6 Season

Package: com.storepilot.db.entities

Seasonal collection period. Dashboard alerts when endDate is within 30 days.

Field Name	Type	Scope	Purpose
id	int	public	Local numeric ID
firestoreId	String	public	Firestore document ID
name	String	public	Season name (e.g. Summer 2025)
startDate	long	public	Season start timestamp (ms)
endDate	long	public	Season end timestamp (ms)
alertDaysBeforeEnd	int	public	Days before end when alert triggers (default 30)
isActive	boolean	public	Whether season is currently active
notes	String	public	Optional notes

4.7 CartItem

Package: com.storepilot.db.entities

One line in a customer cart. Persisted in Firestore for cross-device persistence.

Field Name	Type	Scope	Purpose
id	int	public	Local numeric ID
firestoreId	String	public	Firestore document ID
customerId	int	public	References User.id of cart owner
productId	int	public	References Product.id
quantity	int	public	Units in cart

4.8 SupportMessage

Package: com.storepilot.db.entities

One chat message. Stored in support/{customerId}/messages sub-collection.

Field Name	Type	Scope	Purpose
id	int	public	Local numeric ID
firestoreId	String	public	Firestore document ID
senderId	int	public	References User.id
senderRole	String	public	Role string for chat bubble styling
messageText	String	public	Message content
imageUrl	String	public	Optional image attachment URL
timestamp	long	public	Unix timestamp (ms)
customerId	int	public	Conversation owner
isRead	boolean	public	Read status

Chapter 5: Implementation - Core Modules

5.1 StorePilotApp

Package: com.storepilot | Extends: android.app.Application

Application subclass that runs before any Activity. Performs three startup tasks in onCreate(): initialises Firebase SDK, creates the notification channel, and schedules the hourly AlarmManager alarm.

scheduleLowStockAlarm(context) - key logic

```
AlarmManager am = (AlarmManager) context.getSystemService(ALARM_SERVICE);
Intent intent = new Intent(context, LowStockReceiver.class);
PendingIntent pi = PendingIntent.getBroadcast(
    context, 0, intent,
    PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE);
am.setInexactRepeating(
    AlarmManager.RTC_WAKEUP,
    System.currentTimeMillis() + AlarmManager.INTERVAL_HOUR,
    AlarmManager.INTERVAL_HOUR, pi);
```

5.2 SessionManager

Package: com.storepilot.core | Pattern: Thread-safe Singleton

Holds the in-memory User object for the duration of the app session. All permission checks and role-dependent UI logic query this class.

Field Name	Type	Scope	Purpose
instance	SessionManager	private static	Singleton instance (lazy, synchronized)
loggedInUser	User	private	Currently authenticated user object

getInstance(): Returns singleton. Synchronized for thread safety.

setLoggedInUser(u): Stores user after successful login.

getUserRole(): Returns role string or null.

logout(): Clears user; calls FirebaseAuth.signOut().

5.3 PermissionManager

Package: com.storepilot.core | Pattern: Pure static utility

Maps permission constants to allowed role lists via a static Map. Cannot be modified at

runtime.

hasPermission(role, perm): Looks up perm in permissionMap; returns true if role is in the list.

currentUserHasPermission(perm): Reads role from SessionManager and delegates to hasPermission().

5.4 FirestoreManager

Package: com.storepilot.core | Pattern: Static helper

Provides imperative save/delete methods for User, Product, Task, Sale, Order. Each method builds a Map<String,Object> and calls Firestore set()/delete(). Used alongside Repositories for compatibility.

5.5 LowStockReceiver

Package: com.storepilot.core | Extends: BroadcastReceiver

Triggered by AlarmManager hourly. Queries Firestore for products with quantity <= 5. If found, fires a notification. The static checkNow(context, testMode) can be called from the UI for manual testing.

```
public static void checkNow(Context context, boolean testMode) {
    FirebaseFirestore.getInstance()
        .collection("products")
        .whereLessThanOrEqualTo("quantity", LOW_STOCK_THRESHOLD)
        .get()
        .addOnSuccessListener(snapshots -> {
            int count = snapshots != null ? snapshots.size() : 0;
            if (count > 0)
                NotificationHelper.sendLowStockNotification(context, count);
            else if (testMode)
                NotificationHelper.sendLowStockNotification(context, 1);
        });
}
```

5.6 NotificationHelper

Package: com.storepilot.core

Creates the HIGH-importance notification channel (API 26+) and posts the low-stock notification. CHANNEL_ID = "storepilot_low_stock", NOTIFICATION_ID = 1001 (fixed so repeated alerts update the same notification).

5.7 CryptoUtil

Package: com.storepilot.auth

PBKDF2WithHmacSHA256 password hashing. ITERATIONS=65536, KEY_LENGTH=256 bits, SALT_BYTES=16. generateSalt() uses SecureRandom. hashPassword() returns Base64-encoded hash. verifyPassword() performs constant-time comparison.

5.8 BaseActivity & FirebaseAuthHelper

BaseActivity (com.storepilot.core): extends AppCompatActivity. Adds checkPermission(perm) and hideViewIfUnauthorized(view, perm). All staff Activities extend BaseActivity.

FirebaseAuthHelper (com.storepilot.auth): wraps FirebaseAuth behind AuthCallback (onSuccess(uid) / onFailure(error)). Provides signUp(), signIn(), signOut(), and getCurrentUid().

Chapter 6: Implementation - Repositories

Each Repository is constructed with an Application parameter. The constructor registers a real-time Firestore snapshot listener that feeds a MutableLiveData, ensuring the ViewModel always has the latest data without polling.

6.1 ProductRepository

Firestore collection: "products"

Snapshot listener ordered by createdAt. getLowStockCountSync() blocks with CountdownLatch for background-thread use (called from StorePilotApp).

getAllProducts():	LiveData via snapshot listener.
getLowStockProducts(thresh):	One-shot query: quantity <= threshold.
getLowStockCountSync(t):	Synchronous version using CountdownLatch.
insert / update / delete:	CRUD via Firestore; update/delete fall back to localId query if firestoreId missing.

6.2 UserRepository

Firestore collection: "users"

Document ID = Firebase UID for direct reads. Synchronous lookup methods use CountdownLatch (called on background thread via AppDatabase.dbExecutor).

getAllUsers():	LiveData via snapshot listener.
insert / update:	Writes document by user.uid.
findById(uid):	Synchronous direct document fetch.
findByEmail(email):	Synchronous whereEqualTo("email").
findByUsername(n):	Synchronous whereEqualTo("username").
getUserCountSync():	Synchronous total user count.

6.3 OrderRepository

Firestore collection: "orders"

placeOrder() uses WriteBatch to atomically create order + items, decrement product quantities, and clear cart items. updateOrderStatus() auto-creates a Sale when status = DELIVERED.

placeOrder(...):	Atomic WriteBatch across 4 collections.
updateOrderStatus():	Updates status; auto-creates Sale on DELIVERED.

getActiveOrders():	Snapshot listener excluding DELIVERED/CANCELLED.
getOrdersByCustomer():	Snapshot filtered by customerId.
getTodayRevenue():	One-shot sum of DELIVERED order totals since startOfDay.
getTodayOrderCount():	One-shot count of orders since startOfDay.

6.4 SaleRepository

Firestore collection: "sales"

Snapshot listener ordered by saleDate. All revenue aggregation methods delegate to the private sumSalesInRange(start, end) helper.

getAllSales():	LiveData via snapshot listener.
getTodaySalesTotal(s,e):	Sum totalPrice between two timestamps.
getSalesTotalByWeek(ref):	Sum of last 7 days.
getSalesTotalByMonth():	Sum of last 30 days.
insert / update / delete:	Standard CRUD.

6.5 TaskRepository

Firestore collection: "tasks"

Snapshot listener ordered by createdAt. getPendingTaskCount() is used by DashboardFragment to show the user's pending task count.

getTasksByUser(uid):	Snapshot listener filtered by assignedTo.
getTeamTasks():	One-shot: isPrivate == false.
getPrivateTasks(uid):	One-shot: isPrivate == true AND createdBy == userId.
getPendingTaskCount(u):	One-shot: status == TODO AND assignedTo == userId.
insert / update / delete:	Standard CRUD.

6.6 SeasonRepository

Firestore collection: "seasons"

Snapshot listener ordered by startDate. getSeasonsEndingSoon() queries endDate between now and now+30 days.

getAllSeasons():	LiveData via snapshot listener.
getSeasonsEndingSoon(now):	One-shot query for near-end seasons.

insert / update / delete: Standard CRUD.

6.7 SupportRepository

Firestore collection: "support"

Messages stored in support/{customerId}/messages sub-collection. The parent document is created/merged on every sendMessage() so getConversationCustomerIds() can list active conversations.

sendMessage():	Creates parent doc (merge) + adds message.
getMessagesForCustomer(cid):	Snapshot listener on messages sub-collection; sorted by timestamp in Java.
getConversationCustomerIds():	Snapshot listener on top-level "support" collection.
markAllAsRead(cid):	Batch-updates isRead=true on unread messages.
getUnreadCount(cid):	Snapshot listener counting isRead=false messages.

Chapter 7: Implementation - ViewModels & UI

7.1 AuthViewModel & LoginActivity

`AuthViewModel` (`com.storepilot.viewmodels`) extends `AndroidViewModel`. Exposes `loginSuccess` (`MutableLiveData<Boolean>`) and `loginError` (`MutableLiveData<String>`). `login()` calls Firebase Auth on the main thread, then fetches the user profile on a background thread via `AppDatabase.dbExecutor`, posting results via `postValue()`.

```
public void login(String email, String password) {
    FirebaseAuth.getInstance()
        .signInWithEmailAndPassword(email, password)
        .addOnSuccessListener(result -> {
            String uid = result.getUser().getUid();
            AppDatabase.dbExecutor.execute(() -> {
                User user = userRepository.findByUid(uid);
                if (user == null) {
                    loginError.postValue("Account not found.");
                    return;
                }
                SessionManager.getInstance().setLoggedInUser(user);
                loginSuccess.postValue(true);
            });
        })
        .addOnFailureListener(e ->
            loginError.postValue("Incorrect email or password."));
}
```

`LoginActivity` observes `loginSuccess` to route to `MainActivity` (staff) or `CustomerMainActivity` (customer). Observes `loginError` to show a Toast.

7.2 ProductViewModel & ProductListFragment

`ProductViewModel` wraps all `ProductRepository` operations. `ProductListFragment` observes `getAllProducts()` and passes the list to `ProductListAdapter` (`RecyclerView`). The FAB (+) is hidden when the user lacks `MANAGE_PRODUCTS` permission. Item taps push `ProductDetailsFragment` onto the back stack via `FragmentManager`.

7.3 OrderViewModel & CheckoutFragment

`OrderViewModel` exposes `orderPlaced` (`MutableLiveData<Boolean>`). `CheckoutFragment` caches the cart items and product list from `LiveData`. On "Place Order" tap: validates address and cart, maps radio button to payment method string, calls `orderViewModel.placeOrder()`. Observes `orderPlaced` to navigate to `OrderConfirmationFragment`.

Checkout atomic sequence

```
User: fills address, selects payment, taps "Place Order"
-> orderViewModel.placeOrder(custId, method, address, cart, products)
-> OrderRepository.placeOrder(..., onSuccess)
-> Firestore.collection("orders").add(order)
-> WriteBatch:
    set orderRef.collection("items").doc = OrderItem
    update products/{firestoreId}.quantity -= qty
    delete carts/{custId}/items/product_{id}
-> batch.commit()
-> onSuccess.run()
-> orderPlaced.postValue(true)
-> Fragment: navigate to OrderConfirmationFragment
```

7.4 DashboardFragment

The staff home screen. Uses five ViewModels simultaneously in onCreateView():

SeasonViewModel -> getSeasonsEndingSoon(now) -> season alert banner

SaleViewModel -> getTodaySalesTotal(s, e) -> today revenue card

ProductViewModel-> getLowStockProducts(10) -> low-stock count card

TaskViewModel -> getPendingTaskCount(userId)-> pending tasks card

OrderViewModel -> getTodayOrderCount(startOfDay) -> orders today card

All five observations use getViewLifecycleOwner() for automatic cleanup.

Chapter 8: Screen Flow & Navigation

8.1 Staff Navigation

```
App Launch -> WelcomeActivity
+-- [Login] -> LoginActivity
|
|           +-- [staff role] -> MainActivity
|           |
|           |   Bottom Nav: Dashboard | Products | Sales | Tasks | Seasons
|           |
|           |   Overflow:  Orders | Support | Admin* | [TEST] Notification | Logout
|           |
|           |   * OWNER only
+-- [Register] -> RoleSelectionActivity -> RegisterActivity
                (first owner only)      -> SetupActivity
```

8.2 Customer Navigation

```
LoginActivity -> [CUSTOMER] -> CustomerMainActivity
Bottom Nav: Home | Cart | Favorites | Orders | Chat
Home        -> CustomerHomeFragment -> ProductDetailFragment
Cart        -> CartFragment -> CheckoutFragment -> OrderConfirmationFragment
Favorites   -> FavoritesFragment -> ProductDetailFragment
Orders      -> OrderHistoryFragment
Chat        -> SupportChatFragment
```

8.3 Registration & Authentication Flow

```
RegisterActivity.attemptRegister():
1. Validate form (fullName, username, email, phone, password, confirm)
2. Map role name to constant (Customer->CUSTOMER, Manager->STORE_MANAGER, etc.)
3. FirebaseAuthHelper.signUp(email, password, username)
   -> Firebase Auth: createUserWithEmailAndPassword()
   -> updateProfile(displayName=username)
   -> callback.onSuccess(uid)
4. Create User POJO with uid from Firebase
5. UserRepository.insert(user) -> Firestore "users/{uid}"
6. Navigate to LoginActivity
```

Chapter 9: Testing

9.1 Manual Testing Approach

Tested on Android emulator (Pixel 6, API 34) and a physical Android 11 device. Each user role tested independently to verify role-based access.

9.2 Test Cases

TC-01: Registration

Steps: Register with each role. Verify Firestore document created with correct role.

Result: PASS - All roles created correctly in Firestore "users" collection.

TC-02: Login routing

Steps: Login as staff -> MainActivity. Login as Customer -> CustomerMainActivity.

Result: PASS - Role-based routing works correctly.

TC-03: Permission - Add Product

Steps: Login as Employee. Navigate Products tab. Verify FAB is hidden.

Result: PASS - FAB hidden for roles without `MANAGE_PRODUCTS`.

TC-04: Permission - Admin screen

Steps: Login as Store Manager. Open overflow menu. Verify "Admin" is not visible.

Result: PASS - Admin hidden for non-OWNER roles.

TC-05: Low-stock notification

Steps: Reduce product quantity to 3 in Firestore. Tap [TEST] Notification from menu.

Result: PASS - Notification appears in status bar within 2 seconds.

TC-06: Place Order

Steps: Login as Customer. Add 2 items to cart. Complete checkout.

Result: PASS - Order in Firestore "orders". Cart cleared. Manager sees order.

TC-07: Order status update

Steps: Login as Manager. Update order PENDING -> DELIVERED.

Result: PASS - Status updates in real time. Sale record auto-created in "sales".

TC-08: Season alert

Steps: Create season with endDate = today + 10 days.

Result: PASS - Yellow banner appears on Dashboard.

TC-09: Support chat

Steps: Customer sends message. Manager logs in and replies.

Result: PASS - Messages appear in real time on both sides.

TC-10: Real-time sync

Steps: Open app on 2 devices. Add product on device 1.

Result: PASS - Product appears on device 2 within 1 second.

9.3 Known Limitations

- Image upload is URL-based; no Firebase Storage integration.
- Payment methods are labels only; no real payment gateway is integrated.
- Reports and Marketing modules were removed during development.
- Push notifications are Android-only; no email notification support.

Chapter 10: Appendix: Full Source Code Listings

Complete source code for 13 key classes with Javadoc documentation headers. Code is printed verbatim from the final version of the project.

A.1 StorePilotApp.java

```
/**
 * StorePilotApp.java - Package: com.storepilot
 * Application subclass. Runs before any Activity.
 * 1. Initialize Firebase SDK.
 * 2. Create low-stock notification channel (required on API 26+).
 * 3. Schedule hourly AlarmManager alarm for LowStockReceiver.
 */
package com.storepilot;
import android.app.AlarmManager; import android.app.Application;
import android.app.PendingIntent; import android.content.Context;
import android.content.Intent;
import com.google.firebase.FirebaseApp;
import com.storepilot.core.LowStockReceiver;
import com.storepilot.core.NotificationHelper;

public class StorePilotApp extends Application {
    @Override public void onCreate() {
        super.onCreate();
        FirebaseApp.initializeApp(this);
        NotificationHelper.createNotificationChannel(this);
        scheduleLowStockAlarm(this);
    }
    /** Schedules inexact repeating alarm every 1 hour. FLAG_UPDATE_CURRENT is idempotent. */
    public static void scheduleLowStockAlarm(Context context) {
        AlarmManager am = (AlarmManager) context.getSystemService(Context.ALARM_SERVICE);
        if (am == null) return;
        Intent intent = new Intent(context, LowStockReceiver.class);
        PendingIntent pi = PendingIntent.getBroadcast(context, 0, intent,
            PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE);
        am.setInexactRepeating(AlarmManager.RTC_WAKEUP,
            System.currentTimeMillis() + AlarmManager.INTERVAL_HOUR,
            AlarmManager.INTERVAL_HOUR, pi);
    }
}
```

A.2 SessionManager.java

```
/**
 * SessionManager.java - Package: com.storepilot.core
 * Thread-safe singleton holding the in-memory session.
```



```
* Cleared on logout via logout().
*/
package com.storepilot.core;
import com.google.firebase.auth.FirebaseAuth;
import com.storepilot.db.entities.User;

public class SessionManager {
    private static SessionManager instance;
    private User loggedInUser;
    private SessionManager() {}
    /** Returns the singleton. Synchronized for thread safety. */
    public static synchronized SessionManager getInstance() {
        if (instance == null) instance = new SessionManager();
        return instance;
    }
    public void setLoggedInUser(User user) { this.loggedInUser = user; }
    public User getLoggedInUser() { return loggedInUser; }
    public boolean isLoggedIn() { return loggedInUser != null; }
    public String getUserRole() {
        return loggedInUser != null ? loggedInUser.getRole() : null;
    }
    /** Clears local session and signs out of Firebase Authentication. */
    public void logout() {
        loggedInUser = null;
        FirebaseAuth.getInstance().signOut();
    }
}
```

A.3 PermissionManager.java

```
/**
 * PermissionManager.java - Package: com.storepilot.core
 * Pure-static class mapping permission constants to allowed roles.
 * Static initializer populates an immutable map at class load time.
 */
package com.storepilot.core;
import java.util.*; import java.util.Map;

public class PermissionManager {
    public static final String MANAGE_USERS      = "MANAGE_USERS";
    public static final String MANAGE_PRODUCTS  = "MANAGE_PRODUCTS";
    public static final String VIEW_PRODUCTS    = "VIEW_PRODUCTS";
    public static final String CREATE_SALE      = "CREATE_SALE";
    public static final String VIEW_SALES_HISTORY="VIEW_SALES_HISTORY";
    public static final String MANAGE_SEASONS    = "MANAGE_SEASONS";
    public static final String CREATE_PRIVATE_TASK="CREATE_PRIVATE_TASK";
    public static final String VIEW_TEAM_TASKS  = "VIEW_TEAM_TASKS";
    public static final String MANAGE_TASKS     = "MANAGE_TASKS";
    public static final String VIEW_ADMIN       = "VIEW_ADMIN";
}
```

```
public static final String OWNER="OWNER", STORE_MANAGER="STORE_MANAGER";
public static final String SHIFT_MANAGER="SHIFT_MANAGER", EMPLOYEE="EMPLOYEE";

private static final Map<String,List<String>> permissionMap=new HashMap<>();
static {
    permissionMap.put(MANAGE_USERS, Arrays.asList(OWNER));
    permissionMap.put(MANAGE_PRODUCTS,Arrays.asList(OWNER));
    permissionMap.put(VIEW_PRODUCTS,
        Arrays.asList(OWNER,STORE_MANAGER,SHIFT_MANAGER,EMPLOYEE));
    permissionMap.put(CREATE_SALE,
        Arrays.asList(OWNER,STORE_MANAGER,SHIFT_MANAGER,EMPLOYEE));
    permissionMap.put(VIEW_SALES_HISTORY,
        Arrays.asList(OWNER,STORE_MANAGER,SHIFT_MANAGER));
    permissionMap.put(MANAGE_SEASONS,Arrays.asList(OWNER,STORE_MANAGER));
    permissionMap.put(CREATE_PRIVATE_TASK,
        Arrays.asList(OWNER,STORE_MANAGER,SHIFT_MANAGER,EMPLOYEE));
    permissionMap.put(VIEW_TEAM_TASKS,
        Arrays.asList(OWNER,STORE_MANAGER,SHIFT_MANAGER));
    permissionMap.put(MANAGE_TASKS, Arrays.asList(OWNER,STORE_MANAGER));
    permissionMap.put(VIEW_ADMIN, Arrays.asList(OWNER));
}
public static boolean hasPermission(String role, String permission) {
    List<String> allowed = permissionMap.get(permission);
    return allowed != null && allowed.contains(role);
}
public static boolean currentUserHasPermission(String permission) {
    String role = SessionManager.getInstance().getUserRole();
    return role != null && hasPermission(role, permission);
}
}
```

A.4 LowStockReceiver.java

```
/**
 * LowStockReceiver.java - Package: com.storepilot.core
 * BroadcastReceiver triggered by AlarmManager every hour.
 * Queries Firestore for products with quantity <= LOW_STOCK_THRESHOLD.
 * checkNow(context, testMode) allows manual testing from the UI.
 */
package com.storepilot.core;
import android.content.BroadcastReceiver; import android.content.Context;
import android.content.Intent; import android.widget.Toast;
import com.google.firebase.firestore.FirebaseFirestore;

public class LowStockReceiver extends BroadcastReceiver {
    /** Products at or below this quantity trigger a notification. */
    public static final int LOW_STOCK_THRESHOLD = 5;

    @Override public void onReceive(Context context, Intent intent) {
```

```
        checkNow(context, false);
    }

    /**
     * Queries Firestore and fires a low-stock notification.
     * @param testMode If true, sends a demo notification even when no
     *                 real low-stock products exist, and shows a Toast.
     */
    public static void checkNow(Context context, boolean testMode) {
        FirebaseFirestore.getInstance()
            .collection("products")
            .whereLessThanOrEqualTo("quantity", LOW_STOCK_THRESHOLD)
            .get()
            .addOnSuccessListener(snapshots -> {
                int count = snapshots != null ? snapshots.size() : 0;
                if (count > 0) {
                    NotificationHelper.sendLowStockNotification(context, count);
                } else if (testMode) {
                    NotificationHelper.sendLowStockNotification(context, 1);
                    Toast.makeText(context,
                        "No real low-stock items - demo notification sent",
                        Toast.LENGTH_LONG).show();
                }
            })
            .addOnFailureListener(e -> {
                if (testMode) Toast.makeText(context,
                    "Firestore error: " + e.getMessage(),
                    Toast.LENGTH_LONG).show();
            });
    }
}
```

A.5 NotificationHelper.java

```
/**
 * NotificationHelper.java - Package: com.storepilot.core
 * Creates the notification channel and posts low-stock notifications.
 * Channel is created once in StorePilotApp.onCreate().
 */
package com.storepilot.core;

import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.content.Context;
import android.os.Build;
import androidx.core.app.NotificationCompat;
import com.storepilot.R;

public class NotificationHelper {
    public static final String CHANNEL_ID = "storepilot_low_stock";
    private static final int NOTIFICATION_ID = 1001;

    /**
```

```
* Creates a HIGH-importance notification channel.
* Idempotent - safe to call multiple times. No-op below API 26.
*/
public static void createNotificationChannel(Context context) {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        NotificationChannel channel = new NotificationChannel(
            CHANNEL_ID, "Low Stock Alerts",
            NotificationManager.IMPORTANCE_HIGH);
        channel.setDescription("Alerts when products are running low");
        channel.enableVibration(true);
        NotificationManager mgr =
            context.getSystemService(NotificationManager.class);
        if (mgr != null) mgr.createNotificationChannel(channel);
    }
}

/**
 * Posts a low-stock notification.
 * @param count Number of products at or below the threshold.
 */
public static void sendLowStockNotification(Context context, int count) {
    String text = count + " product(s) are running low. Tap to review.";
    NotificationCompat.Builder builder =
        new NotificationCompat.Builder(context, CHANNEL_ID)
            .setSmallIcon(R.drawable.ic_notification)
            .setContentTitle("Low Stock Alert")
            .setContentText(text)
            .setStyle(new NotificationCompat.BigTextStyle().bigText(text))
            .setPriority(NotificationCompat.PRIORITY_HIGH)
            .setAutoCancel(true);
    NotificationManager mgr = (NotificationManager)
        context.getSystemService(Context.NOTIFICATION_SERVICE);
    if (mgr != null) mgr.notify(NOTIFICATION_ID, builder.build());
}
}
```

A.6 CryptoUtil.java

```
/**
 * CryptoUtil.java - Package: com.storepilot.auth
 * PBKDF2WithHmacSHA256 password hashing utility.
 * NIST-recommended parameters: 65536 iterations, 256-bit key, 16-byte salt.
 */
package com.storepilot.auth;
import android.util.Base64;
import java.security.*; import java.security.spec.InvalidKeySpecException;
import javax.crypto.SecretKeyFactory; import javax.crypto.spec.PBEKeySpec;

public class CryptoUtil {
```

```
private static final int ITERATIONS = 65536;
private static final int KEY_LENGTH = 256;
private static final String ALGORITHM = "PBKDF2WithHmacSHA256";
private static final int SALT_BYTES = 16;

/** Generates a cryptographically secure 16-byte random salt (Base64). */
public static String generateSalt() {
    byte[] salt = new byte[SALT_BYTES];
    new SecureRandom().nextBytes(salt);
    return Base64.encodeToString(salt, Base64.NO_WRAP);
}

/** Returns Base64-encoded PBKDF2 hash of password+salt. */
public static String hashPassword(String password, String salt) {
    try {
        byte[] saltBytes = Base64.decode(salt, Base64.NO_WRAP);
        PBEKeySpec spec = new PBEKeySpec(
            password.toCharArray(), saltBytes, ITERATIONS, KEY_LENGTH);
        SecretKeyFactory factory = SecretKeyFactory.getInstance(ALGORITHM);
        byte[] hash = factory.generateSecret(spec).getEncoded();
        spec.clearPassword();
        return Base64.encodeToString(hash, Base64.NO_WRAP);
    } catch (NoSuchAlgorithmException | InvalidKeySpecException e) {
        throw new RuntimeException("Error hashing password", e);
    }
}

/** Verifies password by hashing and comparing with the expected hash. */
public static boolean verifyPassword(
    String password, String salt, String expectedHash) {
    return hashPassword(password, salt).equals(expectedHash);
}
}
```

A.7 User.java

```
/**
 * User.java - Package: com.storepilot.db.entities
 * Data model for all users (staff + customers).
 * Stored in Firestore collection "users", keyed by Firebase UID.
 */
package com.storepilot.db.entities;
import com.google.firebase.firestore.DocumentSnapshot;
import java.util.HashMap; import java.util.Map;

public class User {
    public int id; public String uid, fullName, username, email, phone, role;
    public long createdAt;
    public User() {}
}
```

```
public User(String fullName, String username, String email, String phone,
            String role, long createdAt) {
    this.fullName=fullName; this.username=username; this.email=email;
    this.phone=phone; this.role=role; this.createdAt=createdAt;
    this.id=(int)(createdAt & 0x7FFFFFFF);
}
public Map<String,Object> toMap() {
    Map<String,Object> m=new HashMap<>();
    m.put("localId",id); m.put("uid",uid!=null?uid:"");
    m.put("fullName",fullName!=null?fullName:"");
    m.put("username",username!=null?username:"");
    m.put("email",email!=null?email:"");
    m.put("phone",phone!=null?phone:"");
    m.put("role",role!=null?role:"CUSTOMER");
    m.put("createdAt",createdAt); return m;
}
/** Deserializes a Firestore DocumentSnapshot into a User. */
public static User fromDoc(DocumentSnapshot doc) {
    if (doc==null||!doc.exists()) return null;
    User u=new User(); u.uid=doc.getId();
    u.id=doc.getLong("localId")!=null
        ?doc.getLong("localId").intValue()
        :Math.abs(doc.getId().hashCode());
    u.fullName=doc.getString("fullName");
    u.username=doc.getString("username");
    u.email=doc.getString("email"); u.phone=doc.getString("phone");
    u.role=doc.getString("role");
    u.createdAt=doc.getLong("createdAt")!=null?doc.getLong("createdAt"):0;
    return u;
}
public int getId(){return id;} public String getUid(){return uid;}
public String getFullName(){return fullName!=null?fullName:username;}
public String getUsername(){return username;}
public String getEmail(){return email;} public String getPhone(){return phone;}
public String getRole(){return role;} public long getCreatedAt(){return createdAt;}
}
```

A.8 Product.java

```
/**
 * Product.java - Package: com.storepilot.db.entities
 * Store product. "quantity" triggers low-stock alert when <= 5.
 * "firestoreId" enables efficient direct Firestore updates.
 */
package com.storepilot.db.entities;
import com.google.firebase.firestore.DocumentSnapshot;
import java.util.HashMap; import java.util.Map;

public class Product {
```

```
public int id; public String firestoreId, name, category, size, color, imageUrl;
public int quantity; public double price, costPrice; public long createdAt;
public Product() {}
public Product(String name,String category,String size,String color,
                int quantity,double price,double costPrice,
                String imageUrl,long createdAt) {
    this.name=name; this.category=category; this.size=size;
    this.color=color; this.quantity=quantity; this.price=price;
    this.costPrice=costPrice; this.imageUrl=imageUrl;
    this.createdAt=createdAt; this.id=(int)(createdAt&0x7FFFFFFF);
}
public Map<String,Object> toMap() {
    Map<String,Object> m=new HashMap<>();
    m.put("localId",id); m.put("name",name!=null?name:"");
    m.put("category",category!=null?category:"");
    m.put("size",size!=null?size:""); m.put("color",color!=null?color:"");
    m.put("quantity",quantity); m.put("price",price);
    m.put("costPrice",costPrice); m.put("imageUrl",imageUrl!=null?imageUrl:"");
    m.put("createdAt",createdAt); return m;
}
public static Product fromDoc(DocumentSnapshot doc) {
    if (doc==null||!doc.exists()) return null;
    Product p=new Product(); p.firestoreId=doc.getId();
    p.id=doc.getLong("localId")!=null?doc.getLong("localId").intValue()
        :Math.abs(doc.getId().hashCode());
    p.name=doc.getString("name"); p.category=doc.getString("category");
    p.size=doc.getString("size"); p.color=doc.getString("color");
    p.quantity=doc.getLong("quantity")!=null?doc.getLong("quantity").intValue():0;
    p.price=doc.getDouble("price")!=null?doc.getDouble("price"):0;
    p.costPrice=doc.getDouble("costPrice")!=null?doc.getDouble("costPrice"):0;
    p.imageUrl=doc.getString("imageUrl");
    p.createdAt=doc.getLong("createdAt")!=null?doc.getLong("createdAt"):0;
    return p;
}
public int getId(){return id;} public String getName(){return name;}
public String getCategory(){return category;} public String getSize(){return size;}
public String getColor(){return color;} public int getQuantity(){return quantity;}
public double getPrice(){return price;} public double getCostPrice(){return costPrice;}
public String getImageUrl(){return imageUrl;} public long getCreatedAt(){return createdAt;}
}
```

A.9 ProductRepository.java

```
/**
 * ProductRepository.java - Package: com.storepilot.repositories
 * Single source of truth for product data.
 * Constructor registers a real-time snapshot listener.
 * Synchronous methods block with CountDownLatch for background-thread use.
 */
```

```
package com.storepilot.repositories;
import android.app.Application;
import androidx.lifecycle.LiveData; import androidx.lifecycle.MutableLiveData;
import com.google.firebase.firestore.*;
import com.storepilot.db.entities.Product;
import java.util.*; import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicReference;

public class ProductRepository {
    private final FirebaseFirestore db = FirebaseFirestore.getInstance();
    private final MutableLiveData<List<Product>> productsLiveData =
        new MutableLiveData<>();

    public ProductRepository(Application application) {
        db.collection("products").orderBy("createdAt")
            .addSnapshotListener((snaps, e) -> {
                if (snaps == null) return;
                List<Product> list = new ArrayList<>();
                for (DocumentSnapshot doc : snaps.getDocuments()) {
                    Product p = Product.fromDoc(doc);
                    if (p != null) list.add(p);
                }
                productsLiveData.postValue(list);
            });
    }

    public LiveData<List<Product>> getAllProducts() { return productsLiveData; }

    /** One-shot query for products at or below threshold. */
    public LiveData<List<Product>> getLowStockProducts(int threshold) {
        MutableLiveData<List<Product>> result = new MutableLiveData<>();
        db.collection("products")
            .whereLessThanOrEqualTo("quantity", threshold).get()
            .addOnSuccessListener(snaps -> {
                List<Product> list = new ArrayList<>();
                for (DocumentSnapshot doc : snaps.getDocuments()) {
                    Product p = Product.fromDoc(doc);
                    if (p != null) list.add(p);
                }
                result.postValue(list);
            });
        return result;
    }

    /** Synchronous low-stock count. Blocks current thread - use on background thread only. */
    public int getLowStockCountSync(int threshold) {
        CountDownLatch latch = new CountDownLatch(1);
        AtomicReference<Integer> count = new AtomicReference<>(0);
        db.collection("products")
            .whereLessThanOrEqualTo("quantity", threshold).get()
```



```
        .addOnCompleteListener(task -> {
            if (task.isSuccessful()) count.set(task.getResult().size());
            latch.countDown();
        });
    try { latch.await(10, TimeUnit.SECONDS); }
    catch (InterruptedException ignored) {}
    return count.get();
}

public void insert(Product product) {
    db.collection("products").add(product.toMap())
        .addOnSuccessListener(ref -> product.firestoreId = ref.getId());
}

public void update(Product product) {
    if (product.firestoreId != null && !product.firestoreId.isEmpty())
        db.collection("products").document(product.firestoreId).set(product.toMap());
}

public void delete(Product product) {
    if (product.firestoreId != null && !product.firestoreId.isEmpty())
        db.collection("products").document(product.firestoreId).delete();
}
}
```

A.10 OrderRepository.java

```
/**
 * OrderRepository.java - Package: com.storepilot.repositories
 * Handles order placement and lifecycle.
 * placeOrder() uses WriteBatch for atomic multi-collection write.
 * updateOrderStatus() auto-creates Sale on DELIVERED.
 */
package com.storepilot.repositories;
import android.app.Application;
import androidx.lifecycle.LiveData; import androidx.lifecycle.MutableLiveData;
import com.google.firebase.firestore.*;
import com.storepilot.db.entities.*;
import java.util.*;

public class OrderRepository {
    private final FirebaseFirestore db = FirebaseFirestore.getInstance();
    public OrderRepository(Application application) {}

    /** Atomically places an order, creates items, decrements stock, clears cart. */
    public void placeOrder(int customerId, String paymentMethod,
        String shippingAddress, List<CartItem> cartItems,
        List<Product> products, Runnable onSuccess) {
        double total = 0;
        for (CartItem ci : cartItems)
            for (Product p : products)
```

```
        if (p.getId() == ci.getProductId()) {
            total += p.getPrice() * ci.getQuantity(); break; }

Order order = new Order(customerId, total, "PENDING",
    System.currentTimeMillis(), paymentMethod, shippingAddress);

db.collection("orders").add(order.toMap())
    .addOnSuccessListener(orderRef -> {
        WriteBatch batch = db.batch();
        for (CartItem ci : cartItems)
            for (Product p : products)
                if (p.getId() == ci.getProductId()) {
                    OrderItem item = new OrderItem(
                        Math.abs(orderRef.getId().hashCode()),
                        p.getId(), ci.getQuantity(), p.getPrice());
                    batch.set(orderRef.collection("items").document(),
                        item.toMap());
                    int newQty=Math.max(0,p.getQuantity()-ci.getQuantity());
                    if (p.firestoreId != null)
                        batch.update(db.collection("products")
                            .document(p.firestoreId),"quantity",newQty);
                    batch.delete(db.collection(
                        "carts/"+customerId+"/items")
                        .document("product_"+p.getId()));
                    break;
                }
            batch.commit().addOnSuccessListener(v -> {
                if (onSuccess != null) onSuccess.run();
            });
        });
    });

/** Updates status. Auto-creates Sale record when status becomes DELIVERED. */
public void updateOrderStatus(Order order, String newStatus) {
    if (order.firestoreId == null) return;
    db.collection("orders").document(order.firestoreId)
        .update("status", newStatus);
    if ("DELIVERED".equals(newStatus)) {
        Sale sale = new Sale(null, 1, order.getTotalPrice(),
            System.currentTimeMillis(), null,
            "Order #" + order.getId() + " delivered");
        db.collection("sales").add(sale.toMap());
    }
}

/** Returns active orders (excludes DELIVERED and CANCELLED). */
public LiveData<List<Order>> getActiveOrders() {
    MutableLiveData<List<Order>> liveData = new MutableLiveData<>();
    db.collection("orders").addSnapshotListener((snaps, e) -> {
        List<Order> list = new ArrayList<>();
```

```
        if (snaps != null)
            for (DocumentSnapshot doc : snaps.getDocuments()) {
                Order o = Order.fromDoc(doc);
                if (o != null && !"DELIVERED".equals(o.status)
                    && !"CANCELLED".equals(o.status)) list.add(o);
            }
        liveData.postValue(list);
    });
    return liveData;
}
}
```

A.11 AuthViewModel.java

```
/**
 * AuthViewModel.java - Package: com.storepilot.viewmodels
 * Handles login logic. Uses Firebase Auth then fetches Firestore profile
 * on a background thread via AppDatabase.dbExecutor.
 */
package com.storepilot.viewmodels;
import android.app.Application;
import androidx.annotation.NonNull; import androidx.lifecycle.AndroidViewModel;
import androidx.lifecycle.MutableLiveData;
import com.google.firebase.auth.FirebaseAuth;
import com.storepilot.core.AppDatabase; import com.storepilot.core.SessionManager;
import com.storepilot.db.entities.User; import com.storepilot.repositories.UserRepository;

public class AuthViewModel extends AndroidViewModel {
    private final UserRepository userRepository;
    public final MutableLiveData<String> loginError = new MutableLiveData<>();
    public final MutableLiveData<Boolean> loginSuccess = new MutableLiveData<>();

    public AuthViewModel(@NonNull Application application) {
        super(application);
        userRepository = new UserRepository(application);
    }

    public void login(String email, String password) {
        FirebaseAuth.getInstance().signInWithEmailAndPassword(email.trim(), password)
            .addOnSuccessListener(result -> {
                String uid = result.getUser().getUid();
                AppDatabase.dbExecutor.execute(() -> {
                    User user = userRepository.findByUid(uid);
                    if (user == null) {
                        loginError.postValue("Account not found. Please register.");
                        return;
                    }
                    SessionManager.getInstance().setLoggedInUser(user);
                    loginSuccess.postValue(true);
                });
            });
    }
}
```

```
        });
    })
    .addOnFailureListener(e ->
        loginError.postValue("Incorrect email or password."));
    }
}
```

A.12 LoginActivity.java

```
/**
 * LoginActivity.java - Package: com.storepilot.auth
 * Handles user login. Observes AuthViewModel for success/failure.
 * Routes to MainActivity (staff) or CustomerMainActivity (customer).
 */
package com.storepilot.auth;
import android.content.Intent; import android.os.Bundle;
import android.widget.Button; import android.widget.Toast;
import androidx.lifecycle.ViewModelProvider;
import com.google.android.material.textfield.TextInputEditText;
import com.storepilot.MainActivity; import com.storepilot.R;
import com.storepilot.core.BaseActivity; import com.storepilot.core.SessionManager;
import com.storepilot.customer.CustomerMainActivity;
import com.storepilot.viewmodels.AuthViewModel;

public class LoginActivity extends BaseActivity {
    private TextInputEditText etEmail, etPassword;
    private Button btnLogin;
    private AuthViewModel authViewModel;

    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_login);
        etEmail    = findViewById(R.id.etUsername);
        etPassword = findViewById(R.id.etPassword);
        btnLogin   = findViewById(R.id.btnLogin);
        authViewModel = new ViewModelProvider(this).get(AuthViewModel.class);

        authViewModel.loginSuccess.observe(this, success -> {
            if (Boolean.TRUE.equals(success)) {
                String role = SessionManager.getInstance().getUserRole();
                Intent intent = "CUSTOMER".equals(role)
                    ? new Intent(this, CustomerMainActivity.class)
                    : new Intent(this, MainActivity.class);
                startActivity(intent); finish();
            }
        });
        authViewModel.loginError.observe(this, error -> {
            if (error != null)
                Toast.makeText(this, error, Toast.LENGTH_SHORT).show();
        });
    }
}
```

```
});  
btnLogin.setOnClickListener(v -> {  
    String email    = etEmail.getText() != null  
        ? etEmail.getText().toString().trim() : "";  
    String password = etPassword.getText() != null  
        ? etPassword.getText().toString() : "";  
    if (email.isEmpty() || password.isEmpty()) {  
        Toast.makeText(this,  
            getString(R.string.error_fill_all_fields),  
            Toast.LENGTH_SHORT).show();  
        return;  
    }  
    authViewModel.login(email, password);  
});  
}  
}
```

A.13 DashboardFragment.java

```
/**  
 * DashboardFragment.java - Package: com.storepilot.dashboard  
 * Staff home screen showing 5 real-time KPI cards via 5 ViewModels.  
 * All LiveData observations use getViewLifecycleOwner() for auto-cleanup.  
 */  
package com.storepilot.dashboard;  
import android.os.Bundle; import android.view.*;  
import android.widget.TextView;  
import androidx.annotation.*; import androidx.fragment.app.Fragment;  
import androidx.lifecycle.ViewModelProvider;  
import com.storepilot.R; import com.storepilot.core.SessionManager;  
import com.storepilot.viewmodels.*;  
import java.text.NumberFormat; import java.util.*;  
  
public class DashboardFragment extends Fragment {  
    private TextView tvSeasonAlert, tvTodaySales,  
                    tvLowStockCount, tvPendingTasks, tvTodayOrders;  
    private View cardSeasonAlert;  
  
    @Nullable @Override  
    public View onCreateView(@NonNull LayoutInflater inflater,  
                            @Nullable ViewGroup container,  
                            @Nullable Bundle savedInstanceState) {  
        return inflater.inflate(R.layout.fragment_dashboard, container, false);  
    }  
  
    @Override  
    public void onViewCreated(@NonNull View view,  
                             @Nullable Bundle savedInstanceState) {  
        super.onViewCreated(view, savedInstanceState);  
    }  
}
```

```
cardSeasonAlert = view.findViewById(R.id.cardSeasonAlert);
tvSeasonAlert    = view.findViewById(R.id.tvSeasonAlert);
tvTodaySales     = view.findViewById(R.id.tvTodaySales);
tvLowStockCount  = view.findViewById(R.id.tvLowStockCount);
tvPendingTasks   = view.findViewById(R.id.tvPendingTasks);
tvTodayOrders    = view.findViewById(R.id.tvTodayOrders);

long now = System.currentTimeMillis();
Calendar cal = Calendar.getInstance();
cal.set(Calendar.HOUR_OF_DAY,0); cal.set(Calendar.MINUTE,0);
cal.set(Calendar.SECOND,0); cal.set(Calendar.MILLISECOND,0);
long startOfDay = cal.getTimeInMillis();
long endOfDay   = startOfDay + 86400000L;
int userId = SessionManager.getInstance().getLoggedInUser() != null
    ? SessionManager.getInstance().getLoggedInUser().getId() : 0;

new ViewModelProvider(this).get(SeasonViewModel.class)
    .getSeasonsEndingSoon(now)
    .observe(getViewLifecycleOwner(), seasons -> {
        if (seasons != null && !seasons.isEmpty()) {
            cardSeasonAlert.setVisibility(View.VISIBLE);
            tvSeasonAlert.setText(getString(
                R.string.season_ending_soon, seasons.get(0).getName()));
        } else cardSeasonAlert.setVisibility(View.GONE);
    });

new ViewModelProvider(this).get(SaleViewModel.class)
    .getTodaySalesTotal(startOfDay, endOfDay)
    .observe(getViewLifecycleOwner(), total ->
        tvTodaySales.setText(NumberFormat.getCurrencyInstance(
            Locale.US).format(total != null ? total : 0.0)));

new ViewModelProvider(this).get(ProductViewModel.class)
    .getLowStockProducts(10)
    .observe(getViewLifecycleOwner(), products ->
        tvLowStockCount.setText(String.valueOf(
            products != null ? products.size() : 0)));

new ViewModelProvider(this).get(TaskViewModel.class)
    .getPendingTaskCount(userId)
    .observe(getViewLifecycleOwner(), count ->
        tvPendingTasks.setText(String.valueOf(count != null ? count : 0)));

new ViewModelProvider(this).get(OrderViewModel.class)
    .getTodayOrderCount(startOfDay)
    .observe(getViewLifecycleOwner(), count -> {
        if (tvTodayOrders != null)
            tvTodayOrders.setText(
                String.valueOf(count != null ? count : 0));
    });
```

```
}  
}
```